

Wipe request — end-to-end message flow

This document traces a single wipe request from the user's click on **"Reset aziendale del dispositivo"** on a managed Windows device all the way through to the Microsoft Graph `wipe` action that initiates the destructive operation on Intune, and back to the user-facing result dialog.

It maps every component, every queue hop, every storage write, every cert gate and every Graph permission that participates. Use it as the canonical reference for incident response, capacity sizing, and onboarding.

Snapshot: the trace below mirrors a real, successfully-executed request on `dev: correlationId = ce0b4cd5280b45d6931c5b017cae27ac` (2026-06-11 13:14:38 → 13:16:01 UTC).

1. Logical architecture

```
flowchart LR
    subgraph Device["Windows device (FC1DSK005)"]
        UI["Start Menu<br/>shortcut"]
        LWP["Launch-Wipe.ps1<br/>(user mode)"]
        TSK["InvokeWipe<br/>scheduled task<br/>(SYSTEM)"]
        WRP["Invoke-WipeFromTask.ps1<br/>(SYSTEM wrapper)"]
        CERT["LocalMachine\\My<br/>client cert<br/>EKU=Client Auth"]
        LR1["(last-result.json)"]
        POLL["StatusPoller<br/>scheduled task"]
        DLG["WipeResult dialog"]
    end

    subgraph AzureFront["Azure (public surface, mTLS)"]
        FD["Front Door / TLS<br/>X-ARR-ClientCert"]
        WEB["idactions-web-dev<br/>(Function App)"]
        SBQ1["SB queue<br/>action-requests"]
    end

    subgraph AzureCore["Azure (private dispatcher)"]
        PROC["idactions-proc-dev<br/>(Function App)"]
        SBQ2["SB queue<br/>action-dispatch"]
        SBQ3["SB queue<br/>wipe-action"]
        TBL["(Table:<br/>ActionStatus)"]
        LED["(Blob:<br/>idempotency ledger)"]
        AUD["(Table:<br/>Audit)"]
    end

    subgraph AzurePriv["Azure (privileged executor)"]
```

```

WIPE["idactions-wipe-dev<br/>(Function App)"]
end

subgraph Graph["Microsoft Graph"]
  GD["Devices API<br/>(Device.Read.All)"]
  GM["managedDevices/{id}<br/>/wipe"]
end

UI --> LWP --> TSK --> WRP
WRP -->|"mTLS POST<br/>/api/actions"| FD --> WEB
WEB -->|"DeviceDirectoryResolver<br/>displayName eq"| GD
WEB -->|"enqueue"| SBQ1 --> PROC
PROC -->|"forward<br/>(opaque envelope)"| SBQ2 --> PROC
PROC -->|"capability route<br/>wipe-action"| SBQ3 --> WIPE
WIPE -->|"managedDevice/{id}/wipe"| GM
PROC -->|"status snapshot"| TBL
WEB -->|"idempotency"| LED
WEB --> AUD
PROC --> AUD
WIPE --> AUD
TSK -->|"writes"| LR1
POLL -->|"updates"| LR1
LR1 --> DLG
WRP -->|"PushLaunch nudge"| TSK
WRP --> CERT

```

2. Sequence diagram (single happy path)

```

sequenceDiagram
    autonumber
    participant U as User
    participant L as Launch-Wipe.ps1<br/>(user)
    participant T as InvokeWipe task<br/>(SYSTEM)
    participant W as Invoke-WipeFromTask.ps1
    participant FD as Front Door<br/>(mTLS)
    participant API as Web Func<br/>(idactions-web-dev)
    participant G as Microsoft Graph
    participant SB1 as SB action-requests
    participant P as Proc Func<br/>(idactions-proc-dev)
    participant SB2 as SB action-dispatch
    participant SB3 as SB wipe-action
    participant X as Wipe Func<br/>(idactions-wipe-dev)
    participant T1 as ActionStatus table
    participant LR as last-result.json
    participant D as Result dialog

    U->>L: click "Reset aziendale del dispositivo"
    L->>L: collect device identity<br/>(EntraDevId, EnrollmentId)

```

```

L->>L: generate X-Request-Nonce<br/>(replay defense)
L->>T: Start-ScheduledTask InvokeWipe
T->>W: powershell.exe -File ...<br/>(SYSTEM context)
W->>W: load client cert<br/>(EKU Client Auth)
W->>FD: POST /api/actions<br/>+ X-Request-Nonce<br/>+ mTLS cert
FD->>API: forward<br/>(X-ARR-ClientCert)
API->>API: 1. validate cert chain<br/>(TrustedRoots/Intermediates)
API->>API: 2. replay guard<br/>(nonce store)
API->>API: 3. payload + allowlist<br/>(Actions:AllowedTypes)
API->>G: 4. binding gate<br/>devices?$filter=<br/>displayName eq 'FC1DSK005'
G->>API: device.deviceId = GUID
API->>API: 5. IDOR check<br/>boundDeviceId == payload.entraDeviceId
API->>SB1: enqueue ActionRequestMessage<br/>(opaque payload)
API->>W: 202 Accepted<br/>{status:queued, correlationId}
W->>LR: write {status:ok, correlationId}
W->>T: PushLaunch MDM nudge<br/>(speeds up status pickup)
W->>T: exit 0

SB1->>P: RequestIntakeFunction
P->>T1: upsert PENDING
P->>SB2: enqueue ActionDispatchMessage
SB2->>P: DispatcherRouterFunction
P->>P: resolve IActionRunner<br/>by actionType
P->>SB3: ForwardingRunner.Send<br/>(per-capability queue)
SB3->>X: WipeActionFunction
X->>X: resolve managedDevice<br/>(via Graph)
X->>G: POST managedDevices/{id}/wipe<br/>(keepEnrollmentData=true,<br/>keepUserData=true)
G->>X: 202 Accepted / sync OK
X->>T1: update RUNNING
X->>SB3: complete (lock released)

loop status polling
  P->>G: GET managedDeviceActions<br/>(probe wipe state)
  G->>P: state=Pending|Running|Done
  P->>T1: state-observed / state-changed
end

loop client polling (every Xs)
  L->>API: GET /api/actions/status?id=...<br/>+ mTLS
  API->>T1: read snapshot
  T1->>API: state
  API->>L: 200 {state, lastUpdated}
  L->>LR: update last-result.json
end

LR->>D: show success / error
D->>U: dialog with outcome

```

3. Stage-by-stage detail

3.1 Device — Launch-Wipe.ps1 (user mode)

Aspect	Detail
Entry point	Start Menu shortcut <code>Reset aziendale del dispositivo.lnk</code> (created by <code>Install.ps1</code>)
Execution context	Logged-on user (NOT SYSTEM) — needed to render WPF dialogs
Responsibilities	(a) collect device identity (Entra device id, enrollment id, hostname); (b) generate <code>X-Request-Nonce</code> for replay defense; (c) start the SYSTEM scheduled task <code>InvokeWipe</code> ; (d) poll <code>last-result.json</code> for the wrapper outcome; (e) render confirmation / progress / result dialogs
Breadcrumb	<code>%LOCALAPPDATA%\IntuneWipeClient\Logs\launcher-observation.json</code> (added v1.0.17)

3.2 Device — InvokeWipe scheduled task → Invoke-WipeFromTask.ps1 (SYSTEM)

Aspect	Detail
Trigger	Started on demand by <code>Launch-Wipe</code> (or by an operator via <code>Start-ScheduledTask</code>)
Action line	<code>powershell.exe -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -File "C:\Program Files\IntuneWipeClient\Invoke-WipeFromTask.ps1"</code>
Responsibilities	(a) load the client cert from <code>LocalMachine\My</code> filtered by configured issuer; (b) build the JSON payload; (c) POST <code>/api/actions</code> over mTLS; (d) write the outcome to <code>last-result.json</code> ; (e) send a local MDM <code>PushLaunch</code> nudge so Intune picks up state changes faster
Persistence	<code>C:\ProgramData\IntuneWipeClient\last-result.json</code> (also <code>wrapper-reached.txt</code> and <code>Logs\Task_*.log</code> transcripts via .NET IO for WDAC resilience)
Hard requirement	All source <code>.ps1</code> files MUST be ASCII or UTF-8 with BOM. PS 5.1 invoked as <code>-File</code> reads with system code page (CP1252) when no BOM is present; a single em-dash silently aborts with exit 1. (This was the root cause of the v1.0.15 silent-crash bug — fixed in v1.0.18.)

3.3 Front Door / Function App ingress — mTLS

Aspect	Detail
Resource	<code>idactions-web-dev</code> (Microsoft.Web/sites , Linux Function App)
TLS	mTLS enforced; the platform forwards the client cert as <code>X-ARR-ClientCert</code> header to the worker
Cert acceptance	<code>ClientCert:TrustedCaThumbprints</code> whitelist + <code>TrustedRootCertificates</code> / <code>TrustedIntermediateCertificates</code> for path validation
Configuration source	All <code>ClientCert:*</code> keys live in <code>idactions-appcfg-dev</code> (Azure App Configuration), surfaced via <code>AddIntuneDeviceActionsAppConfig("web")</code> at startup

3.4 Web Function — ActionRequestFunction.cs

The HTTP handler is intentionally capability-agnostic. Order of gates matters; each one fail-closes with a typed `action.denied.*` audit event.

- 1. ClientCertValidator.Validate
 - trust anchor, ECU Client Auth, optional revocation
- 2. ReplayProtector.Check
 - X-Request-Nonce vs nonce store (sliding window)
- 3. Payload + Actions:AllowedTypes allowlist
 - unknown actionType ⇒ 400
- 4. Certificate ⇔ device binding (IDOR defense)
 - ClientCertValidator.GetBoundDeviceId (Auto mode):
 - a) operator thumbprint→GUID map
 - b) SAN URI = GUID literal
 - c) SAN DNS = GUID literal
 - d) Subject CN = GUID literal
 - e) DeviceDirectoryResolver: SAN DNS FQDN → Graph
/devices?\$filter=displayName eq '<host>' → deviceId
 - boundDeviceId == payload.entraDeviceId else 401
- 5. IdempotencyLedger.TryClaim
 - duplicate replay ⇒ 200 (same correlationId)
- 6. ServiceBusSender.Send (action-requests queue)
 - 202 Accepted {status:queued, correlationId}

uamiWeb required Graph permission: `Device.Read.All` (application). Granted via `tools/Grant-GraphPermissions.ps1`. Without it, step 4e fail-closes and ALL on-prem-PKI clients (cert with SAN=DNS FQDN, no embedded GUID) are rejected with a misleading `client certificate is missing the configured device-id binding claim`.

3.5 Proc Function — capability-agnostic dispatcher

Function	Role
RequestIntakeFunction	Consumes <code>action-requests</code> . Persists initial status row (<code>PENDING</code>) to <code>ActionStatus</code> table. Forwards an <code>ActionDispatchMessage</code> to <code>action-dispatch</code> (W3C traceparent restored by <code>ServiceBusTraceContextMiddleware</code>).
DispatchRouterFunction	Consumes <code>action-dispatch</code> . Resolves <code>IActionRunner</code> by <code>actionType</code> via DI. For <code>wipe</code> , dispatches to <code>WipeForwardingRunner</code> which sends to the per-capability <code>wipe-action</code> queue.
ActionStatusPoller	Periodically probes <code>IActionStatusProbe</code> implementations (Wipe, Autopilot, BitLocker) against Graph; emits <code>action.state-observed</code> / <code>action.state-changed</code> audit events; updates <code>ActionStatus</code> table.

3.6 Wipe Function — privileged executor (idactions-wipe-dev)

Aspect	Detail
Trigger	<code>wipe-action</code> SB queue (passwordless, MI-bound)
Identity	<code>idactions-uami-wipe-dev</code> (privileged Graph: <code>DeviceManagementManagedDevices.PrivilegedOperations.All</code> , <code>...Read.All</code> , <code>Device.Read.All</code> , <code>GroupMember.Read.All</code>)
Graph call	<code>POST /deviceManagement/managedDevices/{managedDeviceId}/wipe</code> with <code>keepEnrollmentData=true</code> , <code>keepUserData=true</code> (corporate-data wipe, not full factory reset)
Sync fallback	If async issuance lingers <code>pending</code> , the runner retries up to 3× with 60s delay (<code>wipe.graph.sync-fallback.issued</code>) — normal behaviour, not an error
Audit	<code>wipe.action.consumed</code> , <code>wipe.graph.issued</code> , optional <code>wipe.graph.sync-fallback.issued</code> , <code>wipe.graph.completed</code>

3.7 Status feedback to user

Step	Path
Proc poller observes state	<code>ActionStatus</code> row updated (state column)
Device polls <code>/api/actions/status</code>	Client cert mTLS + IDOR re-check
<code>Launch-Wipe</code> reads <code>last-result.json</code>	Wrapper + poller both append updates; launcher renders progress
Final dialog	<code>WipeResultDialogs.ps1</code> (<code>Show-WipeSuccessDialog</code> / <code>Show-WipeErrorDialog</code> / <code>Show-WipeUnknownDialog</code> with <code>-ReasonHint</code>)

Direct `Start-ScheduledTask InvokeWipe` invocations bypass `Launch-Wipe` entirely — the SYSTEM task runs hidden, has no UI, and `Send-UserNotification` is a no-op stub by design (`msg.exe` / `WTSSendMessage` produce opaque popups and are disabled on Server SKUs). For user feedback, always go through the Start Menu shortcut.

4. Resources touched per request

Layer	Resource (dev)	Purpose
Identity	<code>idactions-uami-web-dev</code>	Graph <code>Device.Read.All</code> (binding resolver) + AppConfig reader + SB Sender on <code>action-requests</code>
Identity	<code>idactions-uami-dev</code>	Proc dispatcher — SB Receiver/Sender + Graph poller

Layer	Resource (dev)	Purpose
Identity	idactions-uami-wipe-dev	Privileged Graph for the wipe executor
Function App	idactions-web-dev	HTTP intake (mTLS)
Function App	idactions-proc-dev	Dispatcher + status poller
Function App	idactions-wipe-dev	Privileged wipe executor
Service Bus	idactions-sb-dev / action-requests	Web → Proc handoff
Service Bus	idactions-sb-dev / action-dispatch	Proc routing fabric
Service Bus	idactions-sb-dev / wipe-action	Proc → Wipe per-capability queue
Storage	idactionsstpdev / Tables	ActionStatus , Audit (cross-account write from Web)
Storage	idactionsstwdev / Blobs	Idempotency ledger
App Configuration	idactions-appcfg-dev	All runtime settings (ClientCert, Actions, Graph, SB, etc.)
App Insights	idactions-ai-dev	customEvents (audit), traces , exceptions , OTel dependencies
Microsoft Graph	devices endpoint	Resolve displayName → deviceId for cert binding (Web)
Microsoft Graph	managedDevices/{id}/wipe	Issue the destructive action (Wipe)

5. Failure modes and their audit fingerprint

Symptom	Audit event	Root cause
400 missing X-Request-Nonce	action.denied.replay	Client didn't generate / send the nonce
401 cert-validation	action.denied.cert-validation	Cert chain / EKU / revocation failure
401 cert-binding-missing	action.denied.cert-binding-missing	Binding resolver returned null. With an FQDN-bearing cert this is almost always a Graph permission gap on uamiWeb (Device.Read.All missing) — confirm with Directory lookup failed for displayName='...'; fail-closed in traces

Symptom	Audit event	Root cause
401 cert-device-mismatch	action.denied.cert-device-mismatch	boundDeviceId != payload.entraDeviceId (cert belongs to a different device)
400 type-not-allowed	action.denied.type-not-allowed	actionType not in Actions:AllowedTypes (App Config)
400 payload-invalid	action.denied.payload-invalid	Missing/invalid entraDeviceId , deviceName , etc.
Silent client crash, no last-result.json , LastTaskResult=1	n/a	Non-ASCII char in a .ps1 saved without BOM → PS 5.1 -File parser crash. Save scripts as ASCII or UTF-8 BOM.
wipe.graph.sync-fallback.issued	(info)	Normal: async wipe lingered pending , runner is retrying. Up to 3 attempts.

6. Worked example — trace from the canonical successful request

```

13:14:38 Web  action.request.received      corr=ce0b4cd5...
13:14:40 Web  action.request.accepted      cert-binding OK (FQDN→GUID via Graph)
13:14:41 Proc  action.dispatch.enqueued
13:14:41 Proc  action.dispatch.received
13:14:41 Proc  action.forwarded             wipe-action queue
13:14:41 Proc  action.dispatch.completed
13:14:41 Wipe  wipe.action.consumed
13:14:45 Wipe  wipe.graph.issued          managedDeviceId=3b6f7b2b-...
13:15:46 Wipe  wipe.graph.sync-fallback.issued attempt 1/3
13:16:01 Proc  action.state-observed        prev=pending
13:16:01 Proc  action.state-changed

```

Total wall-clock from POST to first state transition: **83 seconds**. Device reboot + factory wipe (corp-data) followed.

7. References

- src/Web/Functions/ActionRequestFunction.cs — the HTTP intake and gates
- src/Shared/Services/ClientCertValidator.cs — cert + binding gate
- src/Shared/Services/DeviceDirectoryResolver.cs — FQDN → Entra deviceId lookup
- src/Proc/Functions/RequestIntakeFunction.cs — SB consumer + dispatcher
- src/Capabilities.Wipe/Runners/WipeRunner.cs — privileged executor
- tools/Grant-GraphPermissions.ps1 — UAMI Graph permission grants

- `client/intune-win32-package/source/Invoke-WipeFromTask.ps1` — SYSTEM wrapper
- `client/intune-win32-package/source/Launch-Wipe.ps1` — user-mode launcher
- `infra/main.bicep` — full Azure topology (UAMIs, role assignments, queues, plans)